0924筆記：

## Docker：

### 1. Docker 版本與資訊

```
# 檢查 Docker 版本
docker --version

# 檢查 Docker 系統資訊
docker info

# 檢查 Docker 版本詳細資訊
docker version
```

```
C:\Users\user>docker --version
Docker version 28.4.0, build d8eb465
```

```
C:\Users\user>docker info
Client:
 Version:    28.4.0
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.9.11
    Path:     C:\Program Files\Docker\cli-plugins\do
```

顯示 Docker 系統資訊

### 2. Image（映像檔）管理：可進入 Docker hub 網站挑選 image

```
# 拉取映像檔
docker pull <image_name>:<tag>
docker pull ubuntu:20.04
docker pull nginx:latest
```

拉取最新版本的 nginx 映像檔

```
C:\Users\user>docker pull ubuntu:20.04
20.04: Pulling from library/ubuntu
13b7e930469f: Pull complete
Digest: sha256:8feb4d8ca5354def3d8fce243717141ce31e2c4
Status: Downloaded newer image for ubuntu:20.04
docker.io/library/ubuntu:20.04
```

Docker pull command

```
docker pull xampp/app:7.4
```

或是複製指令到終端机也可以下載

Run in Docker Desktop

若有下載 Docker Desktop 可直接点選，会自動下載

Requires **Docker Desktop 4.37.1** or later.

```
# 列出本地映像檔
docker images
docker image ls
```

```
PS C:\Users\user> docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    54e66cc1dd1f   7 weeks ago    20.3kB
xampp/app     7.4       2cb39980b7f4   2 months ago   2.05GB
ubuntu        20.04     8feb4d8ca535   5 months ago   109MB
```

顯示本地所有的映像檔

```
# 刪除映像檔
docker rmi <image_id>
docker rmi <image_name>:<tag>
```

執行後，xampp/app 被刪除

```
PS C:\Users\user> docker rmi 2cb39980b7f4
Untagged: xampp/app:7.4
Deleted: sha256:2cb39980b7f473a5a14029fcb9396500b4a6634ea7e1b79
PS C:\Users\user> docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hello-world   latest    54e66cc1dd1f   7 weeks ago    20.3kB
ubuntu        20.04     8feb4d8ca535   5 months ago   109MB
```

image id

```
# 搜尋映像檔
docker search <image_name>

# 建立映像檔（從 Dockerfile）
docker build -t <image_name>:<tag> .
```

### 3. 容器（Container）管理

```
# 執行容器
docker run <image_name>
docker run -it ubuntu:20.04 /bin/bash     # 互動模式
docker run -d nginx:latest                # 背景執行
docker run -p 8080:80 nginx:latest        # 埠號對應
```

这样终端机会自行下載最新版本

```
PS C:\Users\user> docker run ubuntu
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
953cdd413371: Pull complete
Digest: sha256:353675e2a41babd526e2b837d7ec780c2a05b
Status: Downloaded newer image for ubuntu:latest
```

```
PS C:\Users\user> docker run ubuntu:20.04
```

需指定要執行哪种参数

```
# 列出執行中的容器
docker ps

# 列出所有容器（包含停止的）
docker ps -a

# 停止容器
docker stop <container_id>
docker stop <container_name>
```

```
# 啟動容器
docker start <container_id>

# 重啟容器
docker restart <container_id>

# 刪除容器
docker rm <container_id>
docker rm <container_name>

# 進入執行中的容器
docker exec -it <container_id> /bin/bash
```

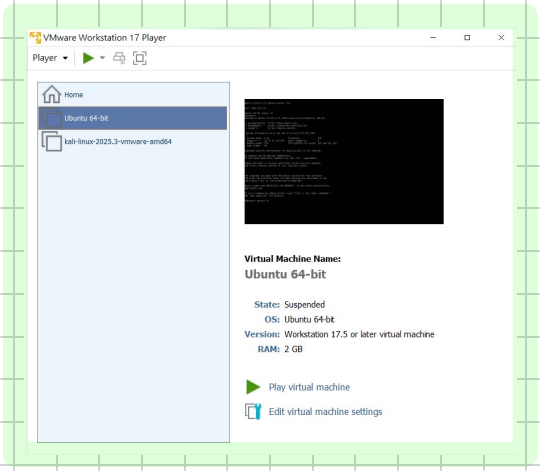
進入 ubuntu 的操作環境

```
PS C:\Users\user> docker run -it ubuntu:20.04 /bin/bash
root@bbf24b540c5a:/# pw
bash: pw: command not found
root@bbf24b540c5a:/# pwd
/
root@bbf24b540c5a:/# ls
bin  boot  dev  etc  home  lib  lib32  lib64  libx32  media
root@bbf24b540c5a:/# exit
```

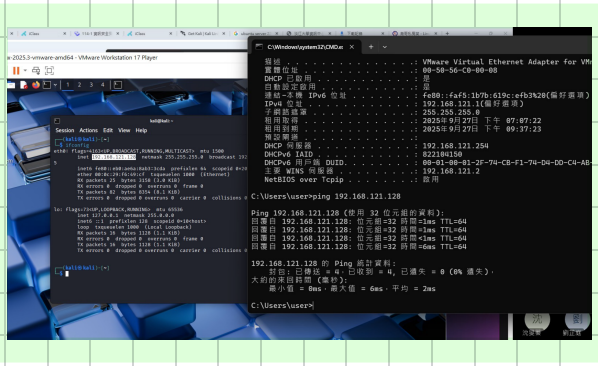
離開操作環境

## VMware：

### # 安裝虛擬机




### # 查看IP位置



0924 筆記。

## Docker

### 4. 常用組合指令

```
# 強制刪除所有停止的容器
docker rm $(docker ps -aq)

# 刪除所有未使用的映像檔
docker image prune

# 系統清理（刪除未使用的容器、映像檔、網路等）
docker system prune

# 查看容器日誌
docker logs <container_id>
docker logs -f <container_id>    # 即時查看

# 複製檔案（容器與主機之間）
docker cp <container_id>:/path/to/file /host/path
docker cp /host/path <container_id>:/path/to/file
```

### 5. Docker Compose 基本指令

```
# 啟動服務（從 docker-compose.yml）
docker-compose up

# 背景啟動服務
docker-compose up -d

# 停止服務
docker-compose down

# 查看服務狀態
docker-compose ps

# 查看服務日誌
docker-compose logs
```

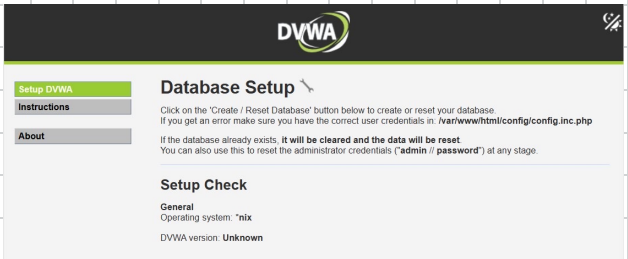
### 6. 實用範例

```
# 執行 Ubuntu 容器並進入 shell
docker run -it --rm ubuntu:20.04 /bin/bash

# 執行 Nginx 網頁伺服器（埠號 8080）
docker run -d -p 8080:80 --name my-nginx nginx:latest

# 執行 MySQL 資料庫
docker run -d -p 3306:3306 --name my-mysql -e MYSQL_ROOT_PASSWORD=mypassword mysql:8.0

# 執行 Python 環境
docker run -it --rm -v $(pwd):/app -w /app python:3.9 python script.py
```

## DVWA (Damn Vulnerable Web Application)

介紹：是一個基於 PHP 和 MySQL 架構的極易受攻擊的網頁應用程式。
這是在合法環境中協助安全專業人士測試其技能和工具，幫助使用者能夠針對一些常見的網頁漏洞進行練習。

```
PS C:\Users\user> git clone https://github.com/digininja/DVWA.git
Cloning into 'DVWA'...
remote: Enumerating objects: 5373, done.
remote: Total 5373 (delta 0), reused 0 (delta 0), pack-reused 5373 (from 1)
Receiving objects: 100% (5373/5373), 2.57 MiB | 5.17 MiB/s, done.
Resolving deltas: 100% (2673/2673), done.
PS C:\Users\user> cd DVWA
PS C:\Users\user\DVWA> docker compose up -d
[+] Running 29/29
 ✔db Pulled
 ✔dvwa Pulled
```




## 安全等級調整

| Level | 描述 |
|---|---|
| Low | 完全沒有任何安全措施，展示因不良編碼習慣而引發的網頁應用程式漏洞，並作為教學或學習基本攻擊技術的平台。 |
| Medium | 展示不良安全實踐的範例，開發者雖嘗試保護應用程式但未成功，並挑戰使用者以提升其攻擊技術。 |
| High | 在中等難度的基礎上，結合更複雜或替代的不良實踐，漏洞可能不允許同樣程度的利用，類似於各種 Capture The Flag (CTF) 比賽。 |
| Impossible | 應對所有漏洞具備防禦能力，用於比較脆弱的源代碼和安全的源代碼。此等級在 DVWA v1.9 之前被稱為 'high'。 |